



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

International Training & Cooperation Institute

Introduction to Blockchain Technology

Lesson 5 - Bitcoin Architecture & Operation



Bitcoin Architecture & Operation



- 1. What is a Bitcoin Transaction**
- 2. Transaction Broadcasting and Memory Pool**
- 3. Bitcoin Ownership Model**
- 4. UTXOs and State Changes**
- 5. Confirmation and Chain Reorganization**
- 6. Summary**
- 7. Discussion**

1. What is a Bitcoin Transaction

Bitcoin: A System with No Balances

- ❖ No **Accounts**: Bitcoin does not have a database that says *Alice: 5 BTC*.
- ❖ UTXOs (Unspent Transaction Outputs): Bitcoin tracks **pieces of money** that have not been spent yet.
- ❖ Analogy: It is like a physical wallet. If you have \$20, you might actually have two \$5 bills and one \$10 bill. To know your total balance, your wallet app must add all these pieces together.

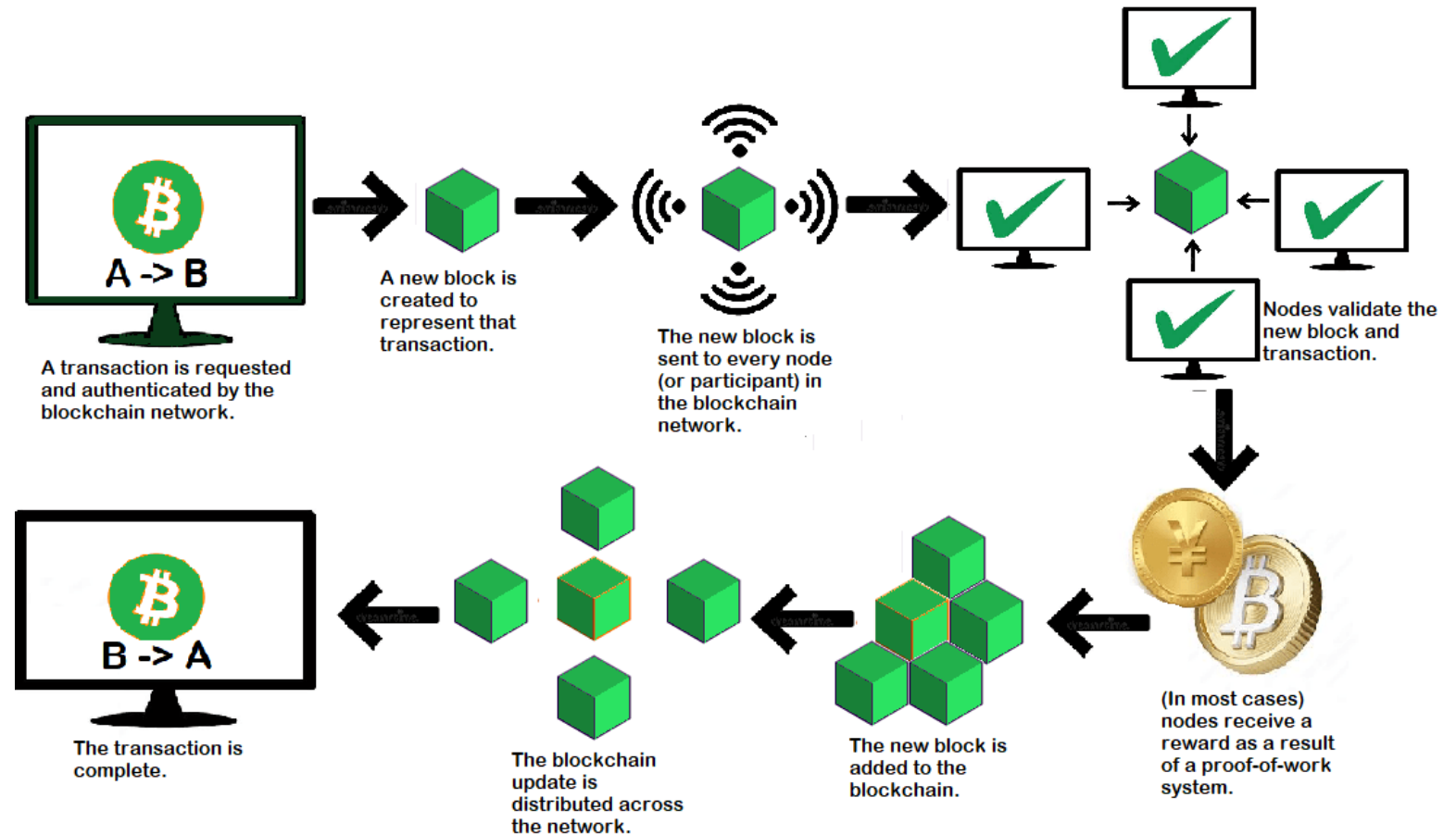
1. What is a Bitcoin Transaction (cont.)

Bitcoin transaction:

- ❖ A data message
- ❖ Created by a user's wallet
- ❖ Used to transfer value
- ❖ Do **not** update balances

Steps:

1. Created by a wallet
2. Broadcast to the network
3. Checked by nodes
4. Temporarily stored
5. Added into a block
6. Confirmed by later blocks



2. Transaction Broadcasting and Memory Pool

Transaction Broadcasting

After creation:

- ❖ The transaction is sent to nearby nodes
- ❖ Each node decides whether to accept it
- ❖ Accepted transactions may be forwarded

Broadcasting:

- ❖ Is peer-to-peer
- ❖ Is not guaranteed

2. Transaction Broadcasting and Memory Pool (cont.)

Temporary Storage of Transactions

Before a transaction is confirmed:

- ❖ It must be stored somewhere
- ❖ Nodes store unconfirmed transactions in memory

This temporary storage is called
Memory Pool (Mempool)

Memory Pool



Blockchain



2. Transaction Broadcasting and Memory Pool (cont.)

Memory Pool

- ❖ Stores unconfirmed transactions
- ❖ Exists only in node memory
- ❖ Is cleared when transactions are confirmed or dropped
- ❖ Each node has its **own mempool**
- ❖ There is no global mempool

***Example:** Node A receives 100 unconfirmed transactions. Node B receives only 80 of them due to network delay. Their memory pools are different, but both follow the same rules.*

Q: Is a transaction lost if it is removed from a memory pool?

Answer: No. It can be rebroadcast later or still exist in other nodes' mempools.

2. Transaction Broadcasting and Memory Pool (cont.)

Why Memory Pools Are Different

- ❖ Nodes receive transactions at different times
- ❖ Nodes may remove old transactions
- ❖ Network delays exist

=> Not all nodes see the same unconfirmed transactions

Q: Does a different memory pool mean nodes disagree on the blockchain?

Answer: No. Mempool differences only affect unconfirmed transactions, not confirmed blocks.

3. Bitcoin Ownership Model

How Ownership Works in Bitcoin

Bitcoin is not a system of accounts and balances

Ownership is defined by:

- ❖ Cryptographic conditions
- ❖ Attached to transaction outputs
- ❖ Only the correct owner can spend them.

3. Bitcoin Ownership Model (cont.)

Transaction Outputs

Each transaction creates one or more **outputs**

Each output:

- ❖ Has a value
- ❖ Has a rule that defines who can spend it

Once created, outputs exist independently

***Example:** Alice has 1 BTC and wants to send 0.4 BTC to Bob. The transaction creates:*

Output 1: 0.4 BTC to Bob

Output 2: 0.6 BTC back to Alice

*Each output can be spent later **independently**.*

4. UTXOs and State Changes

UTXO: Unspent Transaction Output

- ❖ A transaction output that has not been spent yet.
- ❖ Only UTXOs can be spent in new transactions.

4. UTXOs and State Changes (cont.)

UTXO Set

- ❖ The list of all unspent outputs
- ❖ Represents the current state of Bitcoin
- ❖ Bitcoin does **not** store account balances, it stores the UTXO set.
- ❖ Your balance is the sum of all of your UTXOs.

Q: Can a UTXO be spent twice?

Answer: No. Once it is spent, it is removed from the UTXO set

4. UTXOs and State Changes (cont.)

The UTXO Set: How Bitcoin Stays Fast

- ❖ **The Full Ledger:** The whole blockchain is very large (hundreds of gigabytes) and is stored on the hard drive (HDD/SSD).
- ❖ **The UTXO Set:** A small list of all **ready to spend** pieces of money.
- ❖ **Memory Usage:** Full nodes keep this small list in their RAM (computer memory).
- ❖ When a user sends money, the miner checks the RAM list immediately. They do not have to search through blocks from 10 years ago to see if you have the money.

4. UTXOs and State Changes (cont.)

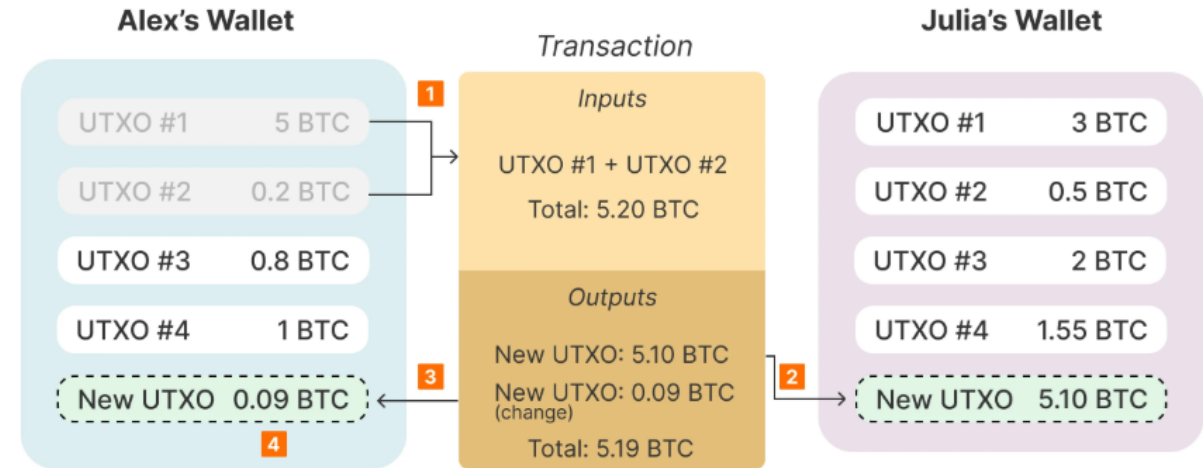
How UTXOs Change

When a transaction is confirmed:

- ❖ Old UTXOs are spent
- ❖ New UTXOs are created

Bitcoin UTXO Model

Alex wants to send 5.10 BTC to Julia



1 Alex's wallet selects the best UTXOs to get to the transaction amount or greater. Since UTXOs are indivisible, the wallet will select two UTXOs.

2 The wallet creates a new UTXO for Julia in the amount of the transaction.

3 The wallet creates a new UTXO for Alex, which is the "change."

4 The transaction fee is not paid to the miner as an output of the transaction. It is inferred by the difference between the value of the inputs and the value of the outputs.

Example:

Block 100: UTXO A exists

Block 101: Transaction spends UTXO A

Block 101: New UTXO B and C are created

State change is **clear and traceable**.

4. UTXOs and State Changes (cont.)

Example: Alice's wallet currently contains three Unspent Transaction Outputs (UTXOs):

- UTXO_1: 1.2 BTC
- UTXO_2: 0.5 BTC
- UTXO_3: 2.0 BTC

Alice wants to buy a computer from Bob for 3.1 BTC. She decides to pay a transaction fee of 0.02 BTC to ensure a fast confirmation.

Tasks:

1. Select the Inputs: Which UTXOs must Alice use to cover the payment and the fee?
2. Calculate the Change: How much BTC will be returned to Alice as a new UTXO?
3. Draw the Transaction: Create a diagram showing the Inputs, Outputs, and the implicit Fee.
4. Final State: List the UTXO set for both Alice and Bob after the transaction is confirmed.

4. UTXOs and State Changes (cont.)

Solution:

1. To pay 3.1 (Price) + 0.02 (Fee) = 3.12 BTC, Alice must select enough UTXOs.
 UTXO_1 (1.2) + UTXO_3 (2.0) = 3.2 BTC (UTXO_2 is not needed and remains in Alice's wallet).

2. Total Inputs: 3.2 BTC

Total Outputs (Payment + Change):

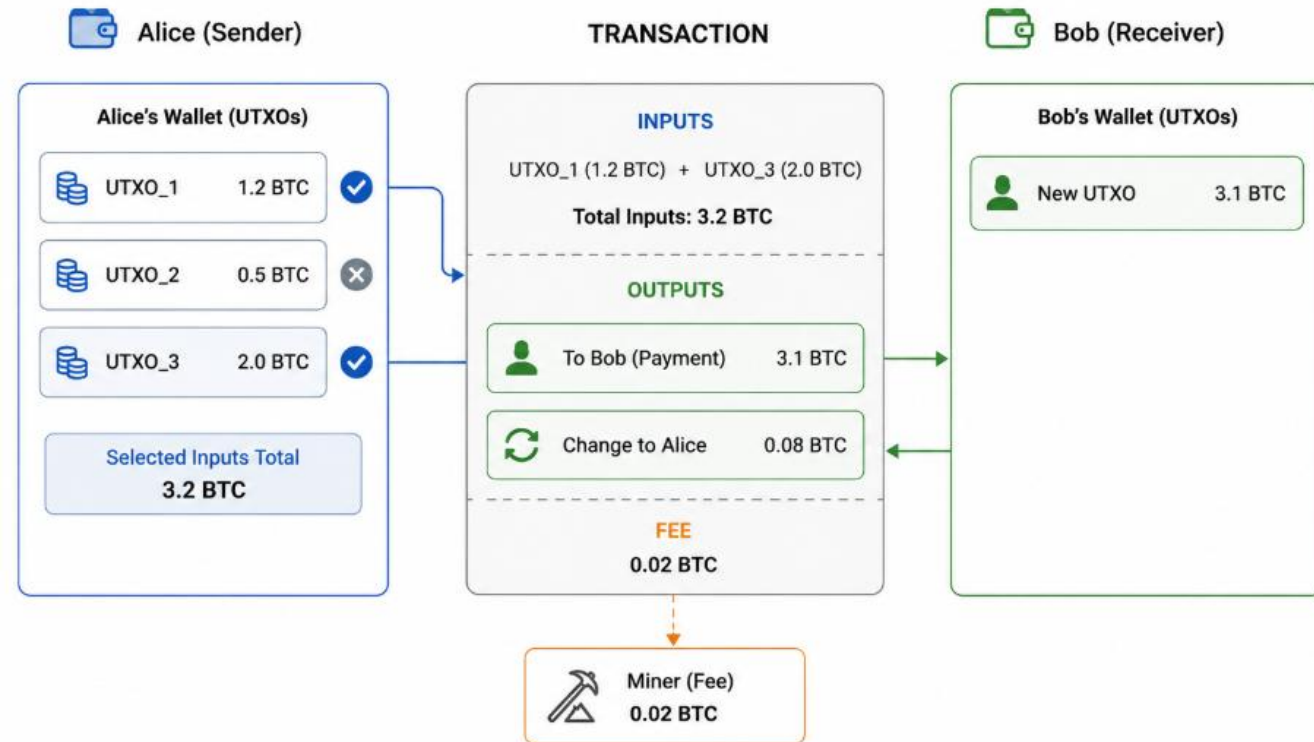
Payment to Bob: 3.1 BTC

Fee: 0.02 BTC

Change to Alice = Total Inputs - Payment - Fee

Change = 3.2 - 3.1 - 0.02 = 0.08 BTC

3. Transaction can be drawn as following:



4. Resulting UTXO Sets:

Alice's UTXO Set: {UTXO_2: 0.5 BTC, New UTXO: 0.08 BTC}

Bob UTXO Set: {New UTXO: 3.1 BTC}.

Miner's Result: The miner who finds the block collects the 0.02 BTC fee via their Coinbase transaction

4. UTXOs and State Changes (cont.)

Transaction Fee: The fee is collected by the miner who includes the transaction

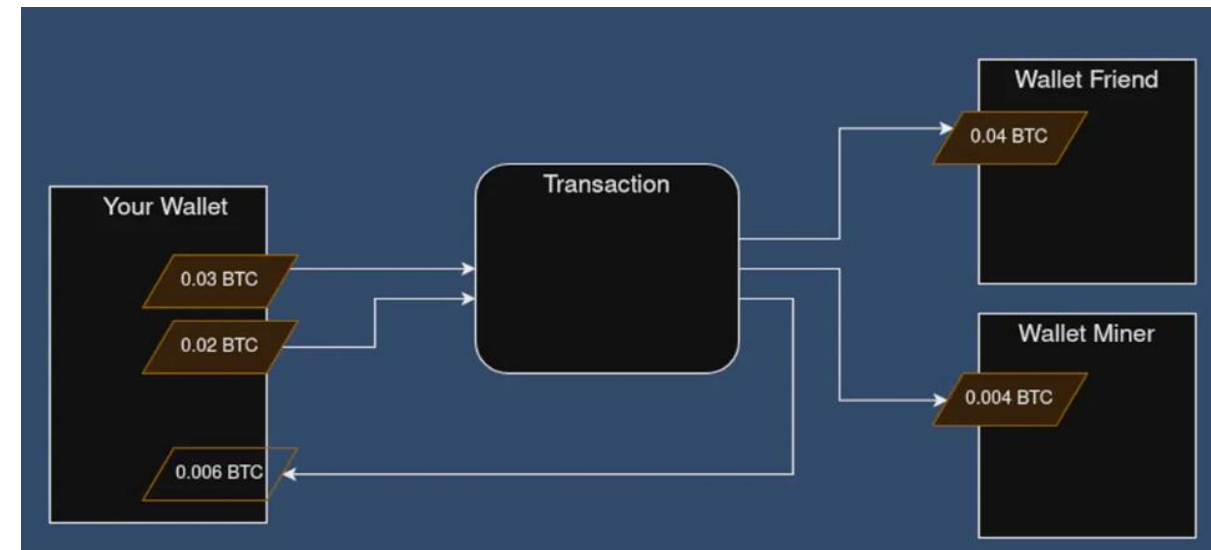
Bitcoin does not calculate transaction fees
The user chooses the fee:

- ❖ Deciding how much value is returned as change.
- ❖ The difference is the transaction fee

Fee = Total inputs – Total outputs

Wallet software usually suggests a fee

- ❖ Higher fee => faster confirmation
- ❖ Lower fee => slower or no confirmation



Example: You have 0.05 BTC and you want to send your friend 0.04 BTC

Input: 0.05 BTC

Outputs: 0.04 BTC + 0.006 BTC

Fee: 0.004 BTC

4. UTXOs and State Changes (cont.)

Transaction Fees (Size vs. Value)

- ❖ While the user chooses the fee, the miner chooses which transactions to include.
- ❖ Miner Logic: Miners want to maximize profit per block. Since a block has a limited size, they calculate the fee-per-byte
- ❖ Fee Rule: You do not pay based on how much money you send. You pay based on the data size (bytes) of the transaction.
- ❖ Heavy Transactions: A transaction with many inputs (many small pieces of money) is **heavy** and costs more.

Example: *Sending 1,000 BTC in one single piece is very cheap. Sending 0.01 BTC made of 100 tiny pieces is very expensive because it takes up more space in the block.*

=> To get a transaction confirmed quickly for the lowest cost, a wallet must balance the total value it leaves as a fee against the total bytes the transaction occupies. This is why selecting fewer UTXOs is the most efficient strategy.

4. UTXO and State Changes (cont.)

Why UTXOs Are Important

UTXOs make it easy to:

- ❖ Detect double spending
- ❖ Check transactions independently
- ❖ Avoid shared balances

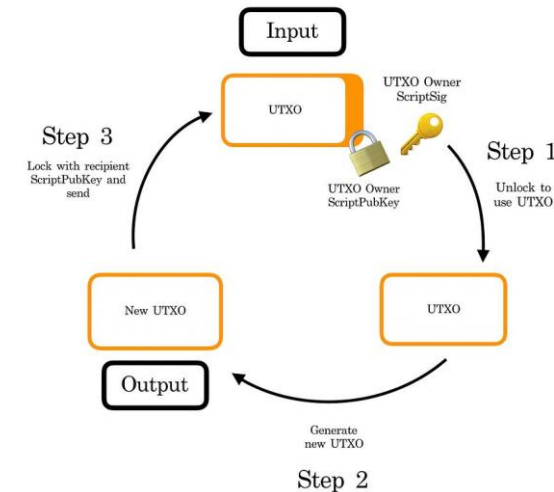
Trade-off:

- ❖ Transactions are harder to construct

4. UTXO and State Changes (cont.)

Bitcoin Script: Digital Locks and Keys

- ❖ More than a transfer: Every transaction piece (UTXO) is protected by a small piece of code called a Script.
- ❖ The **Lock** (Locking Script): The sender sets a condition (e.g., "Only the person with Alice's Private Key can spend this").
- ❖ The **Key** (Unlocking Script): The person spending the money must provide their Digital Signature to prove they own it.



- ❖ Bitcoin Script is intentionally not Turing-complete and does not support general **loops**, helping ensure predictable and bounded execution.

5. Confirmation and Chain Reorganization

Confirmation

A transaction is confirmed when:

- ❖ It is included in a block
- ❖ If another block is added after that:
- ❖ The transaction gets one more confirmation

Example:

A transaction is included in Block #800000 → 1 confirmation

Block #800001 is added → 2 confirmations

Block #800002 is added → 3 confirmations

5. Confirmation and Chain Reorganization (cont.)

Why Multiple Confirmations Matter

Blocks can be:

- ❖ Temporarily replaced
- ❖ Reordered

More confirmations:

- ❖ Make reversal harder
- ❖ Increase confidence

Bitcoin **Finality**:

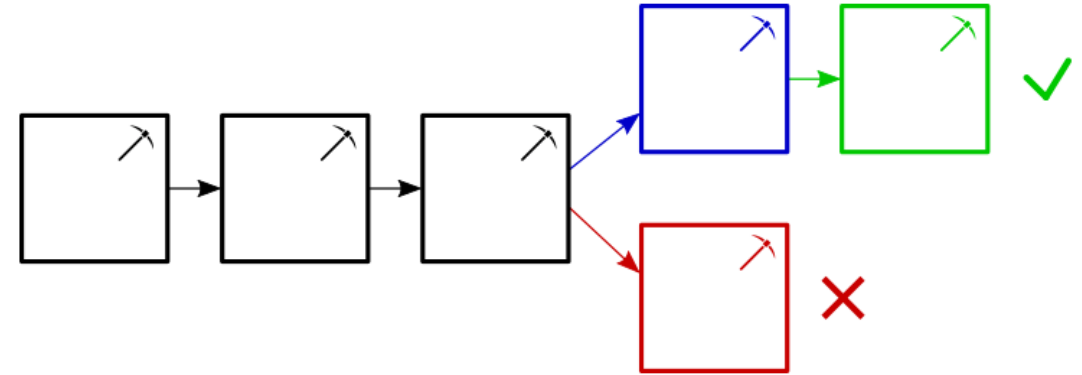
- ❖ Probabilistic
- ❖ Not instant

5. Confirmation and Chain Reorganization (cont.)

Chain Reorganization

Chain reorganization happens when:

- ❖ Two block chains exist
- ❖ One becomes longer
- ❖ Nodes switch to the longer chain
- ❖ Some transactions may become unconfirmed



Q: Is chain reorganization caused by an attack?

Answer: No. Small reorganizations are normal in decentralized networks.

6. Summary

- ❖ Transactions are broadcast and stored temporarily
- ❖ Memory Pools are local to each node
- ❖ Ownership is defined by outputs
- ❖ UTXOs represent system state
- ❖ Confirmations increase confidence

7. Discussion

1. Alice has the following unspent transaction outputs: UTXO1 (3 BTC) and UTXO2 (2 BTC). Alice wants to send 4 BTC to Bob, pay a transaction fee of 0.1 BTC, and receive the remaining change herself. Draw the transaction inputs/outputs. Show the resulting UTXO set after the transaction is confirmed.

2. A full node receives the following transaction:

Input: UTXO = 2 BTC

Outputs:

1.5 BTC → Bob

0.5 BTC → Alice

What will be the transaction fee? Is this a valid transaction? Would a full node accept this transaction?

7. Discussion (cont.)

3. Alice has the following unspent transaction outputs (UTXOs):

- UTXO1: 1 BTC
- UTXO2: 2.1 BTC
- UTXO3: 0.8 BTC
- UTXO4: 4 BTC

Alice wants to send 3 BTC to Bob and pay a transaction fee of 0.2 BTC.

She does not want to spend all her UTXOs in this transaction.

Show two different valid ways Alice can select UTXOs as inputs to construct this transaction.

For each way:

- List the inputs used
- Calculate the total input amount
- Show the transaction outputs
- List Alice's resulting UTXO set after the transaction is confirmed